ResNet for CIFAR-10 with Under 5 Million Parameters

Arjun Bajpai (ab13092)
New York University

Deep Learning, Project 1, Fall 2026

Code: `https://github.com/antailbaxt3r/Deep-Learning-NYU-2026`

1 Overview

The goal of the project is to get the highest test accuracy on CIFAR-10 [11] with a ResNet [7] that has at most 5 million trainable parameters. Pretrained weights and extra data are not allowed. We first fixed a standard, strong training recipe and used it to choose the architecture. Then we tested, one at a time, techniques that other papers report as helpful. All runs were scored on the same held-out images, so each difference comes from the one change being tested.

2 Data and evaluation

The competition gives 50,000 labeled training images of size 32×32 in 10 classes (the standard CIFAR-10 training set) and 2,000 unlabeled test images.

Validation split. We set aside the same 5,000 randomly chosen training images in every run (chosen with random seed 0) and trained on the other 45,000. We made all choices using these 5,000 images only. At 97.5% accuracy, the standard error on 5,000 images is about 0.22 points, so we treat differences smaller than about 0.3 points as ties. Every run trains for a fixed number of epochs with no early stopping. This means the final model can be trained again on all 50,000 images with exactly the same settings.

The test images come from a different collection. The Kaggle test images are the test split of CIFAR-10.2 [13], a newer collection of CIFAR-10 style images gathered from the same source as the original. Earlier work shows that every model scores lower on such new test sets than on the original [17, 13], so we expected the Kaggle score to be well below our validation score. We used this only to understand our scores. We did not use the CIFAR-10.2 labels or its training images, and we did not make any choice based on the Kaggle leaderboard. With 2,000 test images, a score near 93.2% has a sampling error of about ±0.7 points, which is larger than most of the differences we measured.

Table 1: The three candidate models, trained for 200 epochs on 45,000 images with the recipe of Section 4. Validation accuracy uses averaged weights (EMA) and flip test-time augmentation. Times are per epoch on one NVIDIA A100 GPU.

Model	N	B	C	Params	Val. acc. (%)	Time
`resnet18_w42`	4	[2,2,2,2]	[42,84,168,336]	4,815,940	96.34	7.5 s
`resnet20_w64`	3	[3,3,3]	[64,128,256]	4,327,754	97.26	7.1 s
`resnet24_se` (SE)	3	[4,4,3]	[64,128,256]	4,733,610	**97.46**	12.1 s

3 Architecture

Design choices. Every model is a stack of residual blocks that compute $\mathrm{ReLU}(S(x) + F(x))$. Here F is conv, BN, ReLU, conv, BN, and the skip connection S is the identity, or a 1×1 convolution with BN when the shape changes. Using the project's notation, we change the number of stages (residual layers) N, the number of blocks in each stage B_i, and the number of channels in each stage C_i. We keep the convolution kernel size $F_i = 3$ and the skip connection kernel size $K_i = 1$, as in standard ResNets. All models start with a single 3×3 convolution with 64 channels, stride 1 and no max pooling, so the first stage works on the full 32×32 image. Each later stage halves the image size. At the end, average pooling covers the whole final feature map ($P = 8$ for three stages, $P = 4$ for four stages), followed by one linear layer.

Where the parameters go. A 3×3 convolution with C input channels and C output channels has $9C^2$ weights, so the widest stage uses most of the budget. In ResNet-18, the fourth stage (512 channels) holds about three quarters of its 11.2 million parameters. There are two simple ways to get under 5 million: keep four stages and make every stage narrower, or remove the fourth stage and keep the standard widths. We tried both. We also tried a deeper three-stage model with squeeze-and-excitation (SE) blocks [10], which reweight the channels of $F(x)$ before the addition (reduction ratio 16). Table 1 lists the three models. `resnet20_w64` has the same depth and widths as WRN-22-4 [20], but uses the block design given in the project description.

4 Training recipe

Optimizer. We use SGD with Nesterov momentum 0.9 [18], batch size 128, a peak learning rate of 0.1 and weight decay 5×10^{-4}. The learning rate grows linearly for the first 5 epochs [6] and then follows a cosine curve down to zero [12], updated after every batch. The loss is cross-entropy with label smoothing of 0.1 [19].

Data augmentation. Each training image gets a random crop after 4 pixels of reflection padding, a random horizontal flip, the CIFAR-10 AutoAugment policy [2], and Cutout [3], which covers a random 16×16 square with the average pixel color.

Weight averaging and test-time augmentation. We keep an exponential moving average (EMA) of the weights and BatchNorm statistics, with decay 0.999 [16]. All results in this report use the EMA weights. At test time, we average the predictions for each image and its mirror image (flip test-time augmentation, or TTA).

Table 2: Augmentation comparison (`resnet20_w64`, 200 epochs, 45,000 training images).

Augmentation	Val. acc. (%)
Crop, flip, AutoAugment and Cutout (ours)	**97.26**
Crop, flip, TrivialAugment and Cutout	97.00
Crop, flip and AutoAugment, no Cutout	96.66

Implementation and hardware. We used PyTorch with bfloat16 mixed precision, on single NVIDIA A100 40GB GPUs on NYU's HPC cloud burst cluster. One run (RegMixup) used a Google Colab L4 GPU. The cluster often stopped jobs early, so the script saves a checkpoint after every epoch and continues from it when restarted. A 400-epoch run of `resnet20_w64` takes about 50 minutes.

5 Results

5.1 Architecture

Table 1 shows that the narrow four-stage ResNet-18 is clearly worse, even with a similar number of parameters. It is 0.9 points below `resnet20_w64`, about four standard errors. Its fourth stage works on 4×4 feature maps, where extra parameters add little, and its first stage is narrow (42 channels) at full image size. The SE model was 0.2 points ahead at 200 epochs, but took 1.7 times as long to train.

5.2 Data augmentation

Table 2 changes one part of the augmentation at a time. Cutout adds 0.6 points. AutoAugment, whose policy was found by a search on CIFAR-10, beats TrivialAugment [14], which needs no search, by 0.26 points.

5.3 Training length and extra techniques

Training for 400 epochs instead of 200 helped both three-stage models (`resnet20_w64`: from 97.26 to 97.60; `resnet24_se`: from 97.46 to 97.64). At 400 epochs, the SE model is only 0.04 points ahead, which is a tie. Training for 1000 epochs did not help more (97.54). Figure 1 shows why the length of training matters. Most of the gain in validation accuracy comes in the last third of training, when the learning rate becomes small. Training accuracy, measured on augmented images, stays below validation accuracy until the very end.

We then added extra techniques, one at a time, to the 400-epoch `resnet20_w64` baseline (Table 3):

- **SAM** [4] ($\rho = 0.05$): each step uses the gradient at the worst nearby point in weight space.
- **Mixup** [21] ($\alpha = 0.2$): pairs of images and their labels are blended together.
- **RegMixup** [15] ($\alpha = 20$): the normal loss plus an extra mixup loss on blended copies.
- **Distillation** [9, 1]: the target is 0.9 times the average prediction of two teachers (our 400-epoch `resnet24_se` and `resnet20_w64`, trained on the same 45,000 images) plus 0.1 times the true label.
- **Rotation prediction** [5, 8]: a self-supervised task where four extra outputs (1,028 parameters) predict how a copy of each image was rotated, with loss weight 0.5.

Table 3: Extra techniques, each added alone to `resnet20_w64` trained for 400 epochs on 45,000 images (EMA weights, flip TTA). The SE model and the 10-view row use `resnet24_se`. Differences smaller than about 0.3 points are within noise.

Variant	Val. acc. (%)	Change
Baseline (`resnet20_w64`, 400 epochs)	97.60	
`resnet24_se`, 400 epochs	**97.64**	+0.04
With rotation prediction (self-supervised)	97.58	−0.02
With distillation from two teachers of the same size	97.56	−0.04
1000 epochs instead of 400	97.54	−0.06
With RegMixup	97.46	−0.14
With mixup	97.38	−0.22
With SAM	97.32	−0.28
`resnet24_se` with 10 test-time views instead of 2	97.60	−0.04

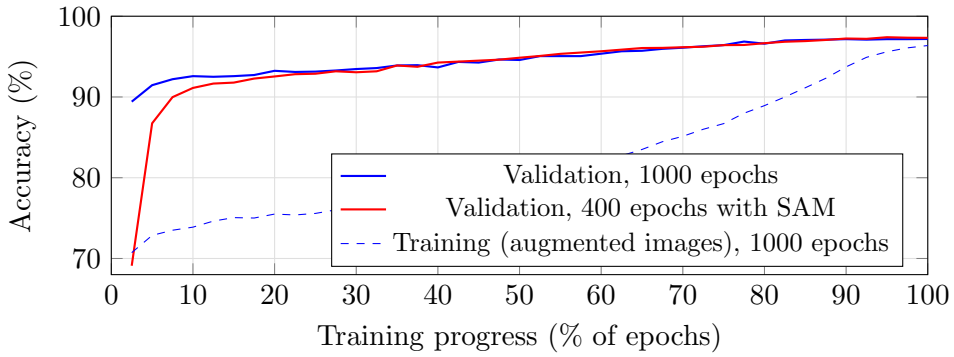


Figure 1: Accuracy of `resnet20_w64` during training, plotted against the share of training completed. Validation curves use EMA weights without TTA.

- **More test-time views**: 10 views (each image and its mirror, at the center and moved 2 pixels in four directions) instead of 2.

None of these techniques beat the baseline. SAM, mixup and RegMixup all lowered accuracy a little. Rotation prediction is a special case: before TTA, its EMA model scored 97.52%, compared with 97.18% for the baseline. After flip TTA, the two were the same (97.58 and 97.60). Flip TTA itself added between 0 and 0.42 points, depending on the run.

5.4 Final model and test accuracy

Our submission is `resnet24_se`, trained again with the same recipe for 400 epochs on all 50,000 training images.

Architecture	ResNet with SE blocks (ratio 16)
Stages and blocks	$N = 3$, $B = [4, 4, 3]$
Channels and kernels	$C = [64, 128, 256]$, $F = 3$, $K = 1$, $P = 8$
Trainable parameters	**4,733,610** (limit: 5,000,000)
Validation accuracy	97.64% (same recipe, 45,000 training images)
Kaggle test accuracy	**0.932**

Our first submission, a 200-epoch model trained on 45,000 images, scored 0.879. `resnet20_w64` trained the same way is just as accurate on validation (97.60%), has 0.4 million fewer parameters, and trains 1.7 times faster. The drop of about 8.6 points from validation to Kaggle comes from the different test collection described in Section 2, not from overfitting. The validation images were never used for training in the runs we compared.

6 What we learned

Spend the budget on width, in three stages. The fourth stage of ResNet-18 holds most of its parameters but works on tiny 4×4 feature maps. Removing it leaves enough parameters for full widths where they matter. Extra depth and SE blocks helped only with shorter training, and made each epoch 70% slower.

Augmentation and training length mattered most. Cutout (+0.6), the augmentation policy (+0.26) and longer training (+0.2 to +0.3) gave the biggest gains.

More regularization did not help. With AutoAugment, Cutout, label smoothing, weight decay and weight averaging together, the model reaches only about 95% training accuracy on augmented images after 400 epochs. We think it is already held back by regularization, which would explain why SAM, mixup and RegMixup lowered accuracy. Teachers of the same size gave the student no new information.

Know what your measurements can show. A fixed validation split let us compare ten changes fairly. The leaderboard, with a sampling error of ±0.7 points, could not have told them apart. But validation images from the original collection cannot measure how a model handles a new collection, which is what RegMixup and rotation prediction aim to improve. Our results may underrate them on the Kaggle test set.

Plan for interrupted jobs. Saving a checkpoint every epoch turned frequent job stops on the shared cluster into short delays instead of lost runs.

Limitations and future work. Each setting was trained once, with one random seed, so differences under 0.3 points are not meaningful. The most promising next step is distillation from a much larger teacher, such as a WRN-28-10 sized network that is used only during training. We implemented it but could not run it before the deadline.

References

[1] L. Beyer, X. Zhai, A. Royer, L. Markeeva, R. Anil, A. Kolesnikov. Knowledge distillation: A good teacher is patient and consistent. *CVPR*, 2022.

[2] E. D. Cubuk, B. Zoph, D. Mané, V. Vasudevan, Q. V. Le. AutoAugment: Learning augmentation strategies from data. *CVPR*, 2019.

[3] T. DeVries, G. W. Taylor. Improved regularization of convolutional neural networks with Cutout. arXiv:1708.04552, 2017.

[4] P. Foret, A. Kleiner, H. Mobahi, B. Neyshabur. Sharpness-aware minimization for efficiently improving generalization. *ICLR*, 2021.

[5] S. Gidaris, P. Singh, N. Komodakis. Unsupervised representation learning by predicting image rotations. *ICLR*, 2018.

[6] P. Goyal et al. Accurate, large minibatch SGD: Training ImageNet in 1 hour. arXiv:1706.02677, 2017.

[7] K. He, X. Zhang, S. Ren, J. Sun. Deep residual learning for image recognition. *CVPR*, 2016.

[8] D. Hendrycks, M. Mazeika, S. Kadavath, D. Song. Using self-supervised learning can improve model robustness and uncertainty. *NeurIPS*, 2019.

[9] G. Hinton, O. Vinyals, J. Dean. Distilling the knowledge in a neural network. arXiv:1503.02531, 2015.

[10] J. Hu, L. Shen, G. Sun. Squeeze-and-excitation networks. *CVPR*, 2018.

[11] A. Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

[12] I. Loshchilov, F. Hutter. SGDR: Stochastic gradient descent with warm restarts. *ICLR*, 2017.

[13] S. Lu, B. Nott, A. Olson, A. Todeschini, H. Vahabi, Y. Carmon, L. Schmidt. Harder or different? A closer look at distribution shift in dataset reproduction. *ICML Workshop on Uncertainty and Robustness in Deep Learning*, 2020.

[14] S. G. Müller, F. Hutter. TrivialAugment: Tuning-free yet state-of-the-art data augmentation. *ICCV*, 2021.

[15] F. Pinto, H. Yang, S.-N. Lim, P. H. S. Torr, P. K. Dokania. Using mixup as a regularizer can surprisingly improve accuracy and out-of-distribution robustness. *NeurIPS*, 2022.

[16] B. T. Polyak, A. B. Juditsky. Acceleration of stochastic approximation by averaging. *SIAM Journal on Control and Optimization*, 30(4), 1992.

[17] B. Recht, R. Roelofs, L. Schmidt, V. Shankar. Do CIFAR-10 classifiers generalize to CIFAR-10? arXiv:1806.00451, 2018.

[18] I. Sutskever, J. Martens, G. Dahl, G. Hinton. On the importance of initialization and momentum in deep learning. *ICML*, 2013.

[19] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, Z. Wojna. Rethinking the Inception architecture for computer vision. *CVPR*, 2016.

[20] S. Zagoruyko, N. Komodakis. Wide residual networks. *BMVC*, 2016.

[21] H. Zhang, M. Cisse, Y. N. Dauphin, D. Lopez-Paz. mixup: Beyond empirical risk minimization. *ICLR*, 2018.